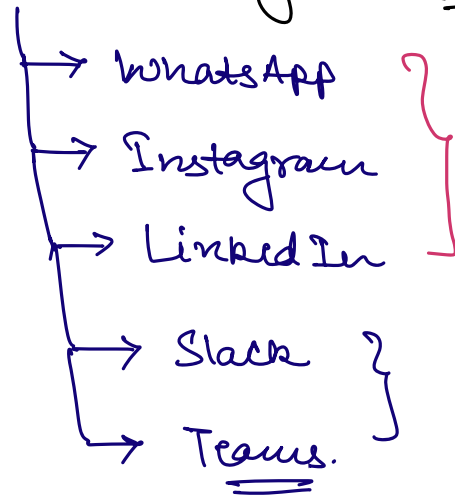


⇒ Design FB Messenger Case Study.



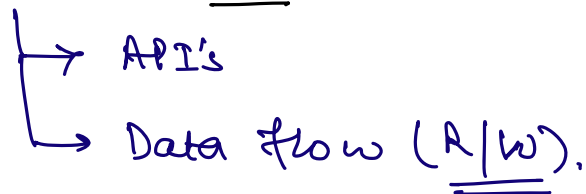
#

1) MVP

2) Scale Estimation

3) Design Tradeoffs.

4) Design Deepdive.



MVP.

→ Send / Receive a msg.

→ Message history.

→ Message → text
→ Media → Image
→ Video
→ file

→ Chat should be realtime.

→ Read receipts.

→ Online / Offline

→ last seen

→ Group Chats.

Scale Estimation.

of Users = 3B

DAU = 1B.

Avg no. of msg per day per user = 20

Total no. of msg per day = 20B

of Writes = 20B / Day.

of Reads = 5x # of Writes.

⇒ Both R+W Heavy.

$$\text{Write Qps} = \frac{20 \times 10^9}{\frac{86400}{10^5}} 10^4 = 200K \text{ qps.}$$

Peak qps = (2-3) x Avg qps.

Storage.



≈ 200B / msg.

1 Day ⇒ 20B × 200 Bytes.

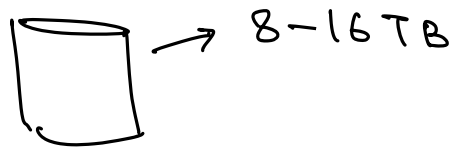
⇒ 4000 GB.

⇒ 4TB.

10 Yrs ⇒ 4TB × 365 × 10

= 16PB.

⇒ SHARDING. ✓



Design Tradeoffs.

High Consistency ✓✓✓ High Availability

Low Latency

⇒ NOSQL.

→ Columnar DB.

↓
Write Heavy.

API's

- `sendMessage(senderId, receiverId, content, msgId)`
- `getConversations(userId, offset, limit)`
- `getMessages(userId, convId, offset, limit)`

Rajneesh



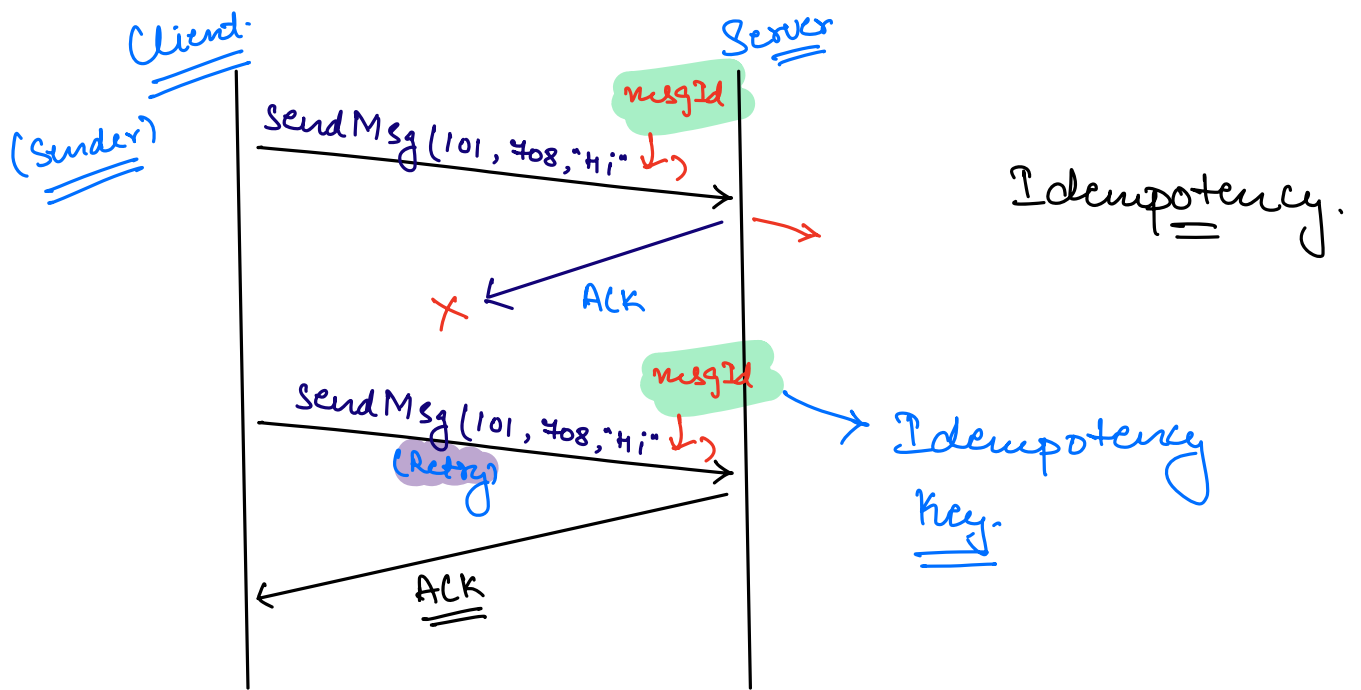
Conversation

1:1
Group.

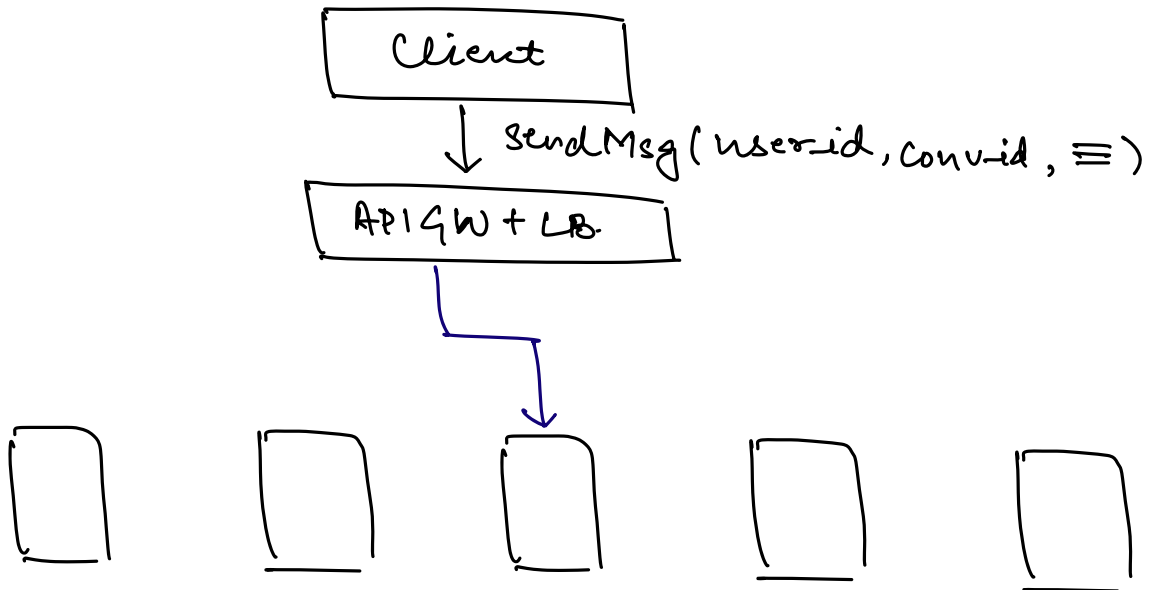
Client
(Sender)

`sendMsg(101, 408, "Hi")` → X

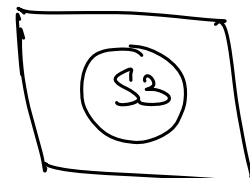
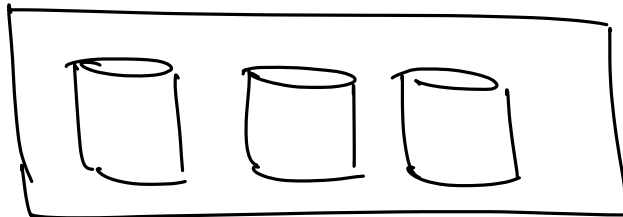
Server



⇒

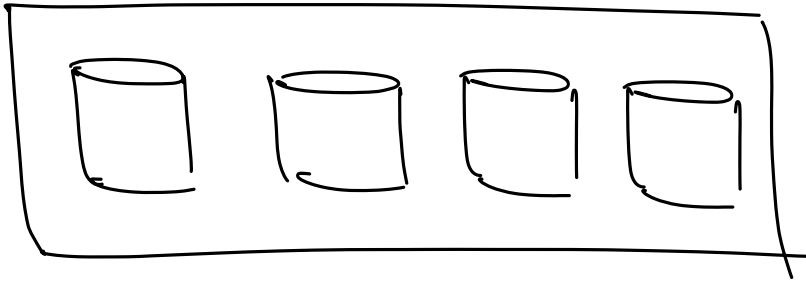


userid.



#

msgs - DB

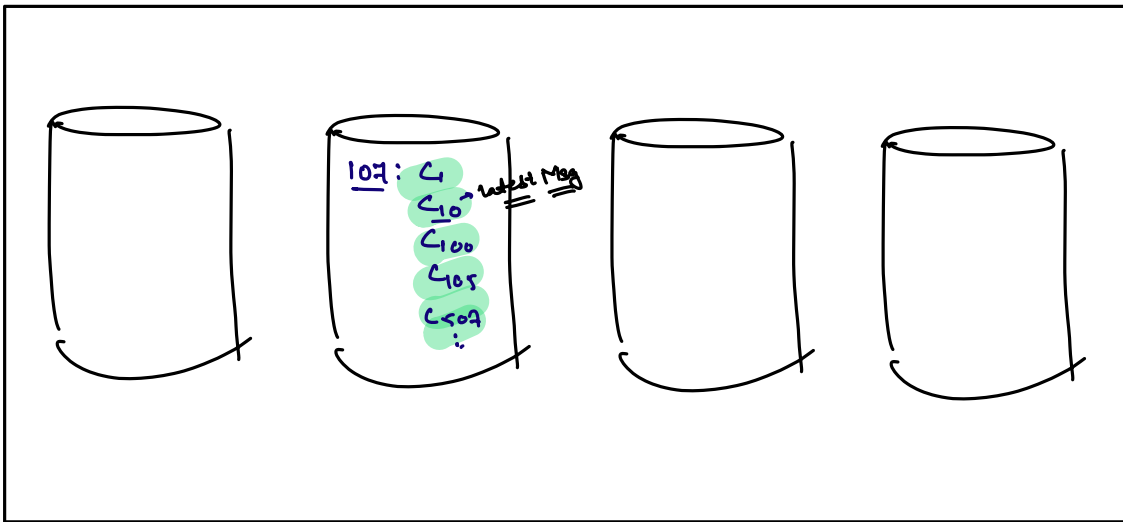


Conversation-id

↳ `getMsgs(convId, limit, -)` ⇒ Intro.

→ `getConversations(userId)` ⇒ Inter.

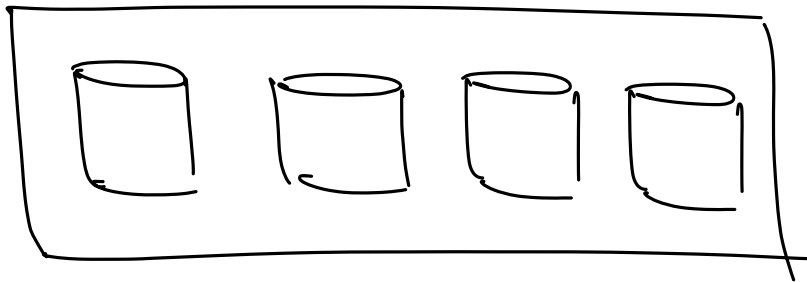
user-id.



Conversations_DB.

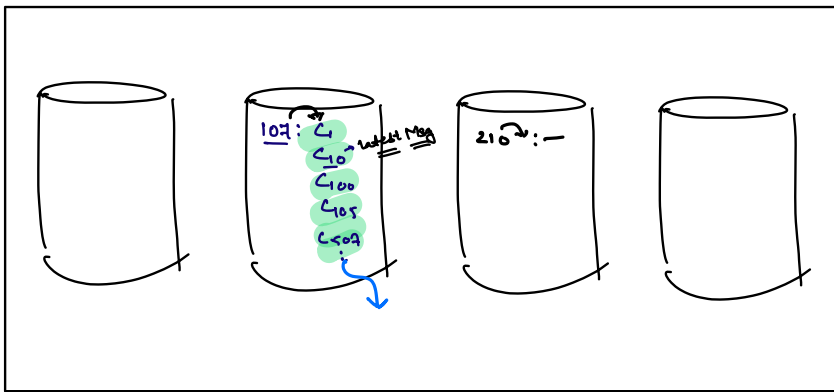
Summary.

msgs - DB



Conversation-id

Conversations_DB.



userid.

hits 1 DB m/c.

↳ `getMsgs(convId, limit, -)` ⇒ Intro.

→ `getConversations(userid)` ⇒ Intro

↳ 1 m/c

→ `sendMsg(senderId, convId, -)`

↓
3 DB m/c

Caching

Msg should be written to DB & Cache in
Synchronized.

→ Write through.